Formal Algorithms for Transformers

Multi-Head Attention, Decoder-Transformer and Decoder-Inference
using
GPT-2 Small Hyperparameters

[Draft 4]

Devi Prasad M

October, 2026

1 The Formal Algorithms for GPT-2 Small Model

The technical report Formal Algorithms for Transformers by Phuong and Hutter of DeepMind (2022, arXiv:2207.09238) provides a formal yet readable breakdown of the original Transformer architecture. By explicitly defining matrix dimensions and tensor shapes in pseudocode, it offers a clear and succinct description of transformer architectures. In this report, we add a few extra annotations to their excellent pseudocode, making it even more accessible to a broader audience. Thiir work provides the baseline for the decoder-only models analyzed in this report.

Phuong and Hutter's pseudocode explicitly shows how a Transformer processes different mathematical objects when predicting the next token. Despite its clarity, it can still be difficult to clearly visualise the concrete shape and structure of the computation. We found that plugging in specific values for all hyperparameters makes these operations much easier to track. Additionally, using an input sequence of a fixed length helps reveal the inner workings and dynamics of the attention mechanism.

In this work, we focus on the following algorithms from the original report, instantiating them with the hyperparameters of OpenAI's GPT-2 Small model. We retain the original algorithm names and numbering scheme for easy cross-referencing:

- Algorithm 1: Token embedding
- Algorithm 3: Basic single-query attention
- Algorithm 4: Attention
- Algorithm 5: MHAttention
- Algorithm 6: layer_norm
- Algorithm 7: Unembedding
- Algorithm 10: DTransformer
- Algorithm 14: DInference

The pseudocode here follows exactly the same syntax as the original. We introduce only one change to the syntax of passing hyperparameter arguments to the algorithms. We will indicate this change when we first encounter the affected code.

2 Hyperparameters of GPT2-small

$\ell_{\max}$	1024	d_{e} (model width)	768
L (layers)	12	d_{mlp} (MLP width)	3072
H (heads)	12	$d_{\mathrm{attn}} = d_{\mathrm{mid}}$ (per head)	64
N_{V} (vocabulary)	50,257	d_{in} (per head)	$= d_{\mathrm{e}} = 768$
total parameters (W_u)	$\approx$ 124.4M	d_{out} (per head)	64

$d_{\mathrm{attn}} = d_{\mathrm{mid}} = d_{\mathrm{e}}/H = 768/12 = 64$: each of the 12 heads works in a 64-dimensional query, key, and value space; $H \cdot d_{\mathrm{mid}} = 768 = d_{\mathrm{e}}$, so the concatenated heads equal the model width before W_o is applied.

3 Notation Simplification

To simplify notation, we denote by $\mathcal{W}$ the entire set of parameters (query, key, value and output linear projections) required by a multi-head attention layer:

$$\mathcal{W} := \begin{pmatrix} \mathbf{W_q^h} \in \mathbb{R}^{d_{\mathrm{attn}} \times d_{\mathrm{x}}},\ \mathbf{b_q^h} \in \mathbb{R}^{d_{\mathrm{attn}}},\ h \in [H] \\ \mathbf{W_k^h} \in \mathbb{R}^{d_{\mathrm{attn}} \times d_z},\ \mathbf{b_k^h} \in \mathbb{R}^{d_{\mathrm{attn}}},\ h \in [H] \\ \mathbf{W_v^h} \in \mathbb{R}^{d_{\mathrm{mid}} \times d_z},\ \mathbf{b_v^h} \in \mathbb{R}^{d_{\mathrm{mid}}},\ h \in [H] \\ \mathbf{W_o} \in \mathbb{R}^{d_{\mathrm{out}} \times H d_{\mathrm{mid}}},\ \mathbf{b_o} \in \mathbb{R}^{d_{\mathrm{out}}} \end{pmatrix} \tag{1}$$

For GPT-2 small ($H = 12$, $d_{\mathrm{attn}} = d_{\mathrm{mid}} = 64$, $d_{\mathrm{x}} = d_z = d_{\mathrm{out}} = 768$), the parameter set $\mathcal{W}$ of a single multi-head attention layer becomes:

$$\mathcal{W} := \begin{pmatrix} \mathbf{W_q^h} \in \mathbb{R}^{64 \times 768},\ \mathbf{b_q^h} \in \mathbb{R}^{64},\ h \in [12] \\ \mathbf{W_k^h} \in \mathbb{R}^{64 \times 768},\ \mathbf{b_k^h} \in \mathbb{R}^{64},\ h \in [12] \\ \mathbf{W_v^h} \in \mathbb{R}^{64 \times 768},\ \mathbf{b_v^h} \in \mathbb{R}^{64},\ h \in [12] \\ \mathbf{W_o} \in \mathbb{R}^{768 \times (12 \cdot 64)} = \mathbb{R}^{768 \times 768},\ \mathbf{b_o} \in \mathbb{R}^{768} \end{pmatrix} \tag{2}$$

Algorithm 3: Basic single-query attention.

Input: $\mathbf{e} \in \mathbb{R}^{d_{\mathrm{in}}}$, vector representation of the current token
Input: $\mathbf{e}_t \in \mathbb{R}^{d_{\mathrm{in}}}$, vector representations of context tokens $t \in [T]$.
Output: $\tilde{\mathbf{v}} \in \mathbb{R}^{d_{\mathrm{out}}}$, vector representation of the token and context combined.
Parameters: $\mathbf{W_q}, \mathbf{W_k} \in \mathbb{R}^{d_{\mathrm{attn}} \times d_{\mathrm{in}}}$, $\mathbf{b_q}, \mathbf{b_k} \in \mathbb{R}^{d_{\mathrm{attn}}}$, the query and key linear projections.
Parameters: $\mathbf{W_v} \in \mathbb{R}^{d_{\mathrm{out}} \times d_{\mathrm{in}}}$, $\mathbf{b_v} \in \mathbb{R}^{d_{\mathrm{out}}}$, the value linear projection.

1 $\mathbf{q} \leftarrow \mathbf{W_q e} + \mathbf{b_q}$
2 $\forall t:\ \mathbf{k}_t \leftarrow \mathbf{W_k e}_t + \mathbf{b_k}$
3 $\forall t:\ \mathbf{v}_t \leftarrow \mathbf{W_v e}_t + \mathbf{b_v}$
4 $\forall t:\ \alpha_t = \dfrac{\exp(\mathbf{q}^\top \mathbf{k}_t / \sqrt{d_{\mathrm{attn}}})}{\sum_u \exp(\mathbf{q}^\top \mathbf{k}_u / \sqrt{d_{\mathrm{attn}}})}$
5 **return** $\tilde{\mathbf{v}} = \sum_{t=1}^{T} \alpha_t \mathbf{v}_t$

4 List of Notation

Symbol	Type	Explanation
$[N]$	$:= \{1, \dots, N\}$	set of integers $1, 2, \dots, N-1, N$
i, j	$\in \mathbb{N}$	generic integer indices
V	$\cong [N_{\mathrm{V}}]$	vocabulary
N_{V}	$\in \mathbb{N}$	vocabulary size
V^*	$= \bigcup_{\ell=0}^{\infty} V^{\ell}$	set of token sequences; elements include e.g. sentences or documents
$\ell_{\max}$	$\in \mathbb{N}$	maximum sequence length
ℓ	$\in [\ell_{\max}]$	length of token sequence
t	$\in [\ell]$	index of token in a sequence
$d_{\dots}$	$\in \mathbb{N}$	dimension of various vectors
$\mathbf{x}$	$\equiv x[1:\ell]$	$\equiv x[1]x[2] \dots x[\ell] \in V^{\ell}$ primary token sequence
$\mathbf{z}$	$\equiv z[1:\ell]$	$\equiv z[1]z[2] \dots z[\ell] \in V^{\ell}$ context token sequence
$M[i, j]$	$\in \mathbb{R}$	entry M_{ij} of matrix $M \in \mathbb{R}^{d \times d'}$
$M[i, :] \equiv M[i]$	$\in \mathbb{R}^{d'}$	i-th row of matrix $M \in \mathbb{R}^{d \times d'}$
$M[:, j]$	$\in \mathbb{R}^{d}$	j-th column of matrix $M \in \mathbb{R}^{d \times d'}$
$\mathbf{e}$	$\in \mathbb{R}^{d_{\mathrm{e}}}$	vector representation / embedding of a token
$\mathbf{X}$	$\in \mathbb{R}^{d_{\mathrm{e}} \times \ell_{\mathrm{x}}}$	encoded primary token sequence
$\mathbf{Z}$	$\in \mathbb{R}^{d_{\mathrm{e}} \times \ell_{\mathrm{z}}}$	encoded context token sequence
Mask	$\in \mathbb{R}^{\ell_{\mathrm{z}} \times \ell_{\mathrm{x}}}$	masking matrix, it determines the attention context for each token
L, L_{enc}, L_{dec}	$\in \mathbb{N}$	number of network (encoder, decoder) layers
l	$\in [L]$	index of network layer
H	$\in \mathbb{N}$	number of attention heads
h	$\in [H]$	index of attention head
N_{data}	$\in \mathbb{N}$	(i.i.d.) sample size
n	$\in [N_{data}]$	index of sample sequence
η	$\in (0, \infty)$	learning rate
τ	$\in (0, \infty)$	temperature; it controls the diversity-plausibility trade-off at inference
$\mathbf{W_e}$	$\in \mathbb{R}^{d_{\mathrm{e}} \times N_{\mathrm{V}}}$	token embedding matrix
$\mathbf{W_p}$	$\in \mathbb{R}^{d_{\mathrm{e}} \times \ell_{\max}}$	positional embedding matrix
$\mathbf{W_u}$	$\in \mathbb{R}^{N_{\mathrm{V}} \times d_{\mathrm{e}}}$	unembedding matrix
$\mathbf{W_q}$	$\in \mathbb{R}^{d_{\mathrm{attn}} \times d_{\mathrm{x}}}$	query weight matrix
$\mathbf{b_q}$	$\in \mathbb{R}^{d_{\mathrm{attn}}}$	query bias
$\mathbf{W_k}$	$\in \mathbb{R}^{d_{\mathrm{attn}} \times d_z}$	key weight matrix
$\mathbf{b_k}$	$\in \mathbb{R}^{d_{\mathrm{attn}}}$	key bias
$\mathbf{W_v}$	$\in \mathbb{R}^{d_{\mathrm{out}} \times d_z}$	value weight matrix
$\mathbf{b_v}$	$\in \mathbb{R}^{d_{\mathrm{out}}}$	value bias
$\mathcal{W}_{\mathbf{qkv}}$		collection of above parameters of a single-head attention layer
$\mathbf{W_o}$	$\in \mathbb{R}^{d_{\mathrm{out}} \times H d_{\mathrm{mid}}}$	output weight matrix
$\mathbf{b_o}$	$\in \mathbb{R}^{d_{\mathrm{out}}}$	output bias
$\mathcal{W}$		collection of above parameters of a multi-head attention layer
$\mathbf{W}_{\mathrm{mlp}}$	$\in \mathbb{R}^{d_1 \times d_2}$	weight matrix corresponding to an MLP layer in a Transformer
$\mathbf{b}_{\mathrm{mlp}}$	$\in \mathbb{R}^{d_1}$	bias corresponding to an MLP layer in a Transformer
γ	$\in \mathbb{R}^{d_{\mathrm{e}}}$	layer-norm learnable scale parameter
β	$\in \mathbb{R}^{d_{\mathrm{e}}}$	layer-norm learnable offset parameter
$\theta, \hat{\theta}$	$\in \mathbb{R}^{d}$	collection of all learnable / learned Transformer parameters

5 Mathematical Conventions of the Report

Phuong and Hutter use the column-vector convention throughout: tokens are *columns* of X and Z, which represent the primary and the context token sequences, respectively. The score matrix is $S = K^\top Q \in \mathbb{R}^{\ell_z \times \ell_x}$ (context length × primary sequence length), with softmax taken down each column. This is the transpose of the row-vector, $S = QK^\top$. Note that the row-softmax convention is more commonly used elsewhere including in the course reference books and in our notes. The two are mathematically identical, just transposed. We retain Phuong and Hutter's convention below since our intention is to match their algorithms exactly.

To begin with consider the expression used to compute an input mask. This appears in many places in the original article:

$$\text{Mask}[t_z, t_x] = \begin{cases} 1, & t_z \le t_x, \\ 0, & t_z > t_x. \end{cases}$$

We may write the same expression in another way to convey the fact that the first subscript identifies rows, and the second, picks columns in a mask matrix:

$$\text{Mask}[r, j] = \begin{cases} 1, & r \le j, \\ 0, & r > j. \end{cases}$$

Recalling that we use the column-vector notation, it is not difficult to see that a tth*column* represents a vector whose first t indices are filled with 1s, followed by zeroes.

r\j	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1	1
2	0	1	1	1	1	1	1	1
3	0	0	1	1	1	1	1	1
4	0	0	0	1	1	1	1	1
5	0	0	0	0	1	1	1	1
6	0	0	0	0	0	1	1	1
7	0	0	0	0	0	0	1	1
8	0	0	0	0	0	0	0	1

In the formal algorithms, $t_x \in [\ell_x]$ indexes positions of the primary sequence $\mathbf{X}$, whose columns are projected to form queries, and $t_z \in [\ell_z]$ indexes positions of the context sequence $\mathbf{Z}$, whose columns are projected to form keys and values.

The mask has shape $\text{Mask} \in \{0,1\}^{\ell_z \times \ell_x}$. If $\text{Mask}[t_z, t_x] = 1$, the query at position t_x may attend to the key–value pair at position t_z. If it is 0, the score $S[t_z, t_x]$ is set to $-\infty$ before softmax, so that pair receives zero attention weight.

In self-attention $\mathbf{Z} = \mathbf{X}$; the causal mask $\text{Mask}[t_z, t_x] = [\![t_z \le t_x]\!]$ allows each query to attend to its own position and all earlier positions.

6 Token Embedding and Unembedding.

The token embedding learns to represent each vocabulary element as a vector in R^{d_e}.

The unembedding learns to convert a vector representation of a token and its context into a distribution over the vocabulary elements. The algorithm describes an independently learned unembedding matrix, but note that sometimes the unembedding matrix is instead fixed to be the transpose of the embedding matrix.

Algorithm 1: Token embedding.

Input: $v \in V \cong [N_V]$, a token ID.
Output: $\mathbf{e} \in \mathbb{R}^{d_e}$, vector representation of the token.
Parameters: $\mathbf{W_e} \in \mathbb{R}^{d_e \times N_V}$, token embedding matrix.

1 **return** $\mathbf{e} = \mathbf{W_e}[:, v]$

Algorithm 7: Unembedding.

Input: $\mathbf{e} \in \mathbb{R}^{d_e}$, a token encoding.
Output: $\mathbf{p} \in \Delta(V)$, a probability distribution over the vocabulary.
Parameters: $\mathbf{W_u} \in \mathbb{R}^{N_V \times d_e}$, the unembedding matrix.

1 **return** $\mathbf{p} = \text{softmax}(\mathbf{W_u e})$

Algorithm 6: $\hat{\mathbf{e}} \leftarrow \texttt{layer_norm}(\mathbf{e}, \gamma, \beta)$

Input : $\mathbf{e} \in \mathbb{R}^{d_e}$, neural network activations.
Output : $\hat{\mathbf{e}} \in \mathbb{R}^{d_e}$, normalized activations.
Parameters : $\gamma, \beta \in \mathbb{R}^{d_e}$, element-wise scale and offset.

1 $\mathbf{m} \leftarrow \sum_{i=1}^{d_e} \mathbf{e}[i]/d_e$

2 $v \leftarrow \sum_{i=1}^{d_e} (\mathbf{e}[i] - \mathbf{m})^2/d_e$

3 **return** $\hat{\mathbf{e}} = \dfrac{\mathbf{e} - \mathbf{m}}{\sqrt{v}} \odot \gamma + \beta$, where $\odot$ denotes element-wise multiplication.

7 Causal (unidirectional/masked) self-attention

Given a sequence of token representations, Algorithm 4 with the causal mask applies attention at every position, treating that position and all preceding ones as its context. Because future positions are masked out, the output at position t depends only on positions $1, \ldots, t$, so this auto-regressive version can be used for online prediction.

Algorithm 4: $\tilde{\mathbf{V}} \leftarrow \textsc{Attention}(\mathbf{X}, \mathbf{Z}, \mathcal{W}_{\mathbf{qkv}}, \text{Mask})$

Input : $\mathbf{X} \in \mathbb{R}^{d_x \times \ell_x}$, $\mathbf{Z} \in \mathbb{R}^{d_z \times \ell_z}$, vector representations of the primary and context sequences.
GPT-2: $\mathbf{X} = \mathbf{Z}$, $d_x = d_z = d_e = 768$, and $\ell_x = \ell_z$.

Output : $\tilde{\mathbf{V}} \in \mathbb{R}^{d_{out} \times \ell_x}$, updated representations of the tokens in $\mathbf{X}$, folding in information from the tokens in $\mathbf{Z}$.
GPT-2: $d_{out} = 64$ (called d_{mid} in Algorithm 5).

Hyperparameters : Mask $\in \{0,1\}^{\ell_z \times \ell_x}$, here the causal mask
$\text{Mask}[t_z, t_x] = [\![t_z \le t_x]\!]$ for $t_z \in [\ell_z]$, $t_x \in [\ell_x]$.

Parameters : $\mathcal{W}_{\mathbf{qkv}}$ consisting of:
$\mathbf{W_q} \in \mathbb{R}^{d_{attn} \times d_x}$, $\mathbf{W_k} \in \mathbb{R}^{d_{attn} \times d_z}$, $\mathbf{W_v} \in \mathbb{R}^{d_{out} \times d_z}$,
$\mathbf{b_q} \in \mathbb{R}^{d_{attn}}$, $\mathbf{b_k} \in \mathbb{R}^{d_{attn}}$, $\mathbf{b_v} \in \mathbb{R}^{d_{out}}$.
GPT-2: $d_{attn} = 64$.

1 $\mathbf{Q} \leftarrow \mathbf{W_q X} + \mathbf{b_q} \mathbf{1}_{\ell_x}^\top$ $\quad [\![\texttt{Query} \in \mathbb{R}^{d_{attn} \times \ell_x}]\!]$

2 $\mathbf{K} \leftarrow \mathbf{W_k Z} + \mathbf{b_k} \mathbf{1}_{\ell_z}^\top$ $\quad [\![\texttt{Key} \in \mathbb{R}^{d_{attn} \times \ell_z}]\!]$

3 $\mathbf{V} \leftarrow \mathbf{W_v Z} + \mathbf{b_v} \mathbf{1}_{\ell_z}^\top$ $\quad [\![\texttt{Value} \in \mathbb{R}^{d_{out} \times \ell_z}]\!]$

4 $\mathbf{S} \leftarrow \mathbf{K}^\top \mathbf{Q} / \sqrt{d_{attn}}$ $\quad [\![\texttt{Score} \in \mathbb{R}^{\ell_z \times \ell_x}]\!]$

5 $\forall t_z, t_x$: **if** $\neg \text{Mask}[t_z, t_x]$ **then** $\mathbf{S}[t_z, t_x] \leftarrow -\infty$

6 **return** $\tilde{\mathbf{V}} = \mathbf{V} \cdot \text{softmax}(\mathbf{S})$ $\quad [\![\tilde{\mathbf{V}} \in \mathbb{R}^{d_{out} \times \ell_x}]\!]$

In GPT-2 small, each layer has $H = 12$ attention heads. Each head reads the 768-dimensional layer-normalized representation of every token in the input sequence and projects it to its own 64-dimensional query, key and value vectors, one of each per token. In self-attention $\ell_x = \ell_z = \ell$, so $\mathbf{Q}^h, \mathbf{K}^h, \mathbf{V}^h \in \mathbb{R}^{64 \times \ell}$. The heads **do not split** the input into twelve 64-dimensional pieces; each applies its own learned projections $\mathbf{W}_\mathbf{q}^h, \mathbf{W}_\mathbf{k}^h, \mathbf{W}_\mathbf{v}^h \in \mathbb{R}^{64 \times 768}$ to the whole representation.

Line 2 of Algorithm 5 calls Algorithm 4 once per head, passing that head's parameters; inside Algorithm 4 we drop the head index h. Lines 1–3 apply the affine map $\mathbf{x} \mapsto \mathbf{W_q x} + \mathbf{b_q}$ to every column of the primary sequence $\mathbf{X}$, giving $d_{attn} = 64$-dimensional queries, and the maps $\mathbf{z} \mapsto \mathbf{W_k z} + \mathbf{b_k}$ and $\mathbf{z} \mapsto \mathbf{W_v z} + \mathbf{b_v}$ to every column of the context sequence $\mathbf{Z}$, giving $d_{attn} = 64$-dimensional keys and $d_{out} = 64$-dimensional values. In GPT-2's self-attention, $\mathbf{Z} = \mathbf{X}$.

Line 4 divides each query–key dot product by $\sqrt{d_{attn}} = 8$, and line 5 sets the scores at masked positions to $-\infty$. In line 6, a softmax over the context positions, taken one column of scores at a time, gives the attention weights of each query, and the product $\mathbf{V} \cdot \text{softmax}(\mathbf{S})$ forms, for each query token, the weighted average of the value vectors under those weights:

$$\tilde{\mathbf{V}}[:, t_x] = \sum_{t_z=1}^{\ell_z} \text{softmax}(\mathbf{S})[t_z, t_x]\, \mathbf{V}[:, t_z].$$

Because the softmax weights are non-negative and sum to 1, each output is a convex combination of the value vectors, and since $\exp(-\infty) = 0$, masked positions receive weight exactly 0.

Each head therefore outputs one 64-dimensional vector per query token. Line 3 of Algorithm 5 stacks the twelve head outputs into a vector of $H\, d_{\text{mid}} = 768$ entries for each query token, forming $\mathbf{Y} \in \mathbb{R}^{768 \times \ell_x}$, and line 4 applies the output projection:

$$\tilde{\mathbf{V}} = \mathbf{W_o}\mathbf{Y} + \mathbf{b_o}\mathbf{1}_{\ell_x}^\top, \qquad \mathbf{W_o} \in \mathbb{R}^{768\times 768},\ \mathbf{b_o} \in \mathbb{R}^{768}.$$

Note the name clash: in Algorithm 5, $d_{\text{out}} = 768$ is the size of this final output, whereas in Algorithm 4 $d_{\text{out}} = 64$ is the size of a single head's output, which Algorithm 5 calls d_{mid}.

The multi-head output $\tilde{\mathbf{V}} \in \mathbb{R}^{768\times \ell_x}$ contains the updated token representations. Each column incorporates information from the positions that token attends to. Under the causal mask, $\tilde{\mathbf{V}}[:, 1{:}t]$ depends only on $\mathbf{X}[:, 1{:}t]$. Every other operation in the network (the embeddings, residual connections, layer normalization, MLP, and unembedding) acts on each column separately. Causal self-attention is thus the only step that mixes information across positions, and each position attends only to itself and earlier positions. Applying this reasoning layer by layer, column t of the final output depends only on the tokens $x[1{:}t]$ and can therefore be used to predict $x[t+1]$.

7.1 Prediction and the Causal Mask

Nothing in the prefix $x[1{:}t]$ determines $x[t+1]$. To "predict" $x[t+1]$ means to estimate its conditional distribution $\hat{P}(x[t{+}1] \mid x[1{:}t])$ from the observed prefix, not to determine it with certainty. The knowledge behind this estimate lies in the trained parameters $\hat{\boldsymbol{\theta}}$, which hold what was learned from the training corpus (for GPT-2, about 8 million web documents, WebText): which tokens tend to follow which contexts, across a very large number of examples. The prefix selects the context while $\hat{\boldsymbol{\theta}}$ supplies what usually comes next in such contexts.

The causal mask ensures that this estimate never uses the answer, or any later token. This matters most in training, where the whole sequence is present in the input and column t could otherwise read $x[t+1]$ directly. At inference there is nothing later to hide, but the mask stays.

Algorithm 5: $\tilde{V} \leftarrow \text{MHAttention}(X, Z, \mathcal{W}, \text{Mask})$

Input : $X \in \mathbb{R}^{d_x \times \ell_x}$ layer-normalized primary sequence.
$Z \in \mathbb{R}^{d_z \times \ell_z}$ layer-normalized context sequence.
$X = Z$, $\ell_x = \ell_z$
Output : $\tilde{V} \in \mathbb{R}^{d_{\text{out}} \times \ell_x}$ attention output.
Hyperparameters : $H = 12$; $\text{Mask} \in \{0,1\}^{\ell_z \times \ell_x}$
Parameters : $\mathcal{W}$ consisting of:
for $h \in [12]$: W^h_{qkv} consisting of:
$\mid W^h_q \in \mathbb{R}^{d_{\text{attn}} \times d_x}$, $b^h_q \in R^{d_{\text{attn}}}$
$\mid W^h_k \in \mathbb{R}^{d_{\text{attn}} \times d_z}$, $b^h_k \in R^{d_{\text{attn}}}$
$\mid W^h_v \in \mathbb{R}^{d_{\text{mid}} \times d_z}$, $b^h_v \in \mathbb{R}^d_{\text{mid}}$
$W_o \in \mathbb{R}^{d_{\text{out}} \times (h \cdot d_{\text{mid}})} = \mathbb{R}^{d_{\text{out}} \times d_{\text{out}}}$, $b_o \in \mathbb{R}^{d_{\text{out}}}$, output projection.

for $h = 1, 2, \ldots, 12$ **do**
  $Y^h \leftarrow \text{Attention}(X, X, W^h_{qkv}, \text{Mask})$ [[`Algorithm 4`; $Y^h \in \mathbb{R}^{64 \times \ell}$]]
$Y \leftarrow [Y^1; Y^2; \ldots; Y^{12}]$ [[`stack heads:` $Y \in \mathbb{R}^{768 \times \ell}$]]
return $\tilde{V} = W_o Y + b_o \mathbf{1}^\top$

Note that a column t of the output matrix $\tilde{V} \in \mathbb{R}^{d_{\text{out}} \times \ell_x}$ is a convex combination of the value vectors $v_{t'}$ over the permitted positions t'.

Decoder-Transformer

Algorithm 10: $P \leftarrow$ DECODER-TRANSFORMER$(x, \boldsymbol{\theta})$

Input : $x \in V^*$, a sequence of token IDs. $length(x) \le \ell_{\max} \le 1024$

Output : $P \in (0,1)^{N_{\mathrm{V}} \times length(x)}$. $N_{\mathrm{V}} = 50257$

Hyperparameters : $\ell_{\max}$, L H, d_{e}, $d_{\mathrm{mlp}} \in \mathbb{N}$
$\ell_{\max} = 1024$, $L = 12$ $H = 12$, $d_{\mathrm{e}} = 768$, $d_{\mathrm{mlp}} = 3072$

Parameters : $\boldsymbol{\theta}$ includes:

$W_e \in \mathbb{R}^{d_{\mathrm{e}} \times N_{\mathrm{V}}}$, token embeddings.

$W_p \in \mathbb{R}^{d_{\mathrm{e}} \times \ell_{\max}}$ positional embeddings.

for $l \in [12]$: hyperparameters of each layer ...

| $\mathcal{W}_l$, attention parameters of Algorithm 5.

| $\gamma_l^{(1)}, \beta_l^{(1)}, \gamma_l^{(2)}, \beta_l^{(2)} \in \mathbb{R}^{d_{\mathrm{e}}}$, two sets of layer-norm parameters.

| $W^l_{\mathrm{mlp1}} \in \mathbb{R}^{d_{\mathrm{mlp}} \times d_{\mathrm{e}}}$, $b^l_{\mathrm{mlp1}} \in \mathbb{R}^{d_{\mathrm{mlp}}}$, MLP parameters.

| $W^l_{\mathrm{mlp2}} \in \mathbb{R}^{d_{\mathrm{e}} \times d_{\mathrm{mlp}}}$, $b^l_{\mathrm{mlp2}} \in \mathbb{R}^{d_{\mathrm{e}}}$, MLP parameters.

$\gamma, \beta \in \mathbb{R}^{d_{\mathrm{e}}}$, final layer-norm parameters.

$W_u \in \mathbb{R}^{N_{\mathrm{V}} \times d_{\mathrm{e}}}$, the unembedding matrix.

$\ell \leftarrow \mathrm{length}(x)$, $\ell_{\mathrm{x}} \leftarrow \mathrm{length}(x)$

for $i \in [\ell]$

  $t \leftarrow x[i]$ [[`next token id`]]

  $e_i \leftarrow W_e[:, t] + W_p[:, i]$ [[`token embedding;` $e_i \in \mathbb{R}^{768}$]]

$X \leftarrow [e_1, e_2, \ldots, e_\ell]$ [[$X \in \mathbb{R}^{768 \times \ell}$]]

for $l = 1, 2, \ldots, 12$ **do**

  for $t \in [\ell]$

    $\tilde{X}[:, t] \leftarrow$ `layer_norm`$(X[:, t] \mid \gamma_l^{(1)}, \beta_l^{(1)})$

  `/* residual attention mixes the unnormalized` x`. */`

  $X \leftarrow X +$ MHATTENTION$(\tilde{X}, \tilde{X}, \mathcal{W}_l, \mathrm{Mask})$ [[`Algorithm 5`]]

  for $t \in [\ell]$

    $\tilde{X}[:, t] \leftarrow$ `layer_norm`$(X[:, t], \gamma_l^{(2)}, \beta_l^{(2)})$

  `/* residual attention` x`. */`

  $X \leftarrow X + W^l_{\mathrm{mlp2}}\,\mathtt{GELU}\left(W^l_{\mathrm{mlp1}}\tilde{X} + b^l_{\mathrm{mlp1}}\mathbf{1}^\top\right) + b^l_{\mathrm{mlp2}}\mathbf{1}^\top_{\ell_{\mathrm{x}}}$ [[$768 \to 3072 \to 768$]]

end

for $t \in [\ell]$

  $X[:, t] \leftarrow$ `layer_norm`$(X[:, t], \gamma, \beta)$

return $P = \mathrm{softmax}(W_u X)$

Decoder-Inference

The hyperparameters $\ell_{\text{gen}} \in \mathbb{N}$ and $\tau \in (0, \infty)$ belong to DInference, not to DTransformer. They are not part of the trained parameters $\hat{\boldsymbol{\theta}}$ and are chosen at inference time.

ℓ_{gen} is the number of tokens generated, and hence the number of calls to DTransformer. Since the last call sees $\ell + \ell_{\text{gen}} - 1$ tokens, we need $\ell + \ell_{\text{gen}} - 1 \leq \ell_{\max}$.

τ is the sampling temperature. Each new token is drawn from $\mathbf{q} \propto \mathbf{p}^{1/\tau}$, where $\mathbf{p}$ is the last column of $\mathbf{P}$. With $\tau = 1$ we sample from $\mathbf{p}$ itself. With $\tau < 1$ the distribution is sharpened, and as $\tau \to 0$ sampling becomes greedy arg max decoding (it concentrates all its mass on the most probable token). With $\tau > 1$ the distribution is flattened, approaching the uniform distribution over tokens as $\tau \to \infty$.

For a fixed input sequence, neither ℓ_{gen} nor τ changes the $\mathbf{P}$ returned by DTransformer. ℓ_{gen} determines how many tokens are generated, and τ how each next token is selected. Because a selected token becomes part of the next input sequence, changing τ (or simply a different random draw) can indirectly change $\mathbf{P}$ at later generation steps.

Algorithm 14: $y \leftarrow \text{DInference}(x, \hat{\theta})$

Input	: $\hat{\theta}$, Trained parameters	(shapes as in Algorithm 10).
Input	: $x \in V^*$,	a prompt, length$(x) \leq 1024$.
Output	: $y \in V^*$,	model's continuation of the prompt.
Hyperparameters	: $\ell_{\text{gen}} \in \mathbb{N}$, $\tau \in (0, \infty)$	inference-time choices.

1 $\ell \leftarrow \text{length}(x)$
2 **for** $i = 1, 2, \ldots, \ell_{gen}$ **do**
3 $\quad P \leftarrow \text{DTransformer}(x, \hat{\theta})$ [[Algorithm 10]]
4 $\quad \mathbf{p} \leftarrow P[:, \ell + i - 1]$
5 $\quad$ sample a token y_i from $\mathbf{q} \propto \mathbf{p}^{1/\tau}$
6 $\quad x \leftarrow [x, y_i]$
7 **return** $y = x[\ell{+}1 : \ell + \ell_{gen}]$

With fixed $\hat{\theta}$, Algorithm 10 is a deterministic function: the same x always gives the same P. Every line in it is a fixed computation with no randomness: embedding lookups, layer norm, matrix products, softmax and GELU.

The explicit randomness in Algorithm 14 is in line 5, where y is sampled from y is sampled from $q \propto p^(1/\tau)$. Therefore, repeated runs from the same prompt can produce altogether different continuations. If we replace sampling with greedy decoding, DInference becomes deterministic for fixed parameters and generation length, provided ties are broken by a fixed rule.

Further, Algorithm 10 has no dropout, and GPT-2's published inference code (found in src/model.py) does not use dropout. In frameworks that include dropout, it is switched off during inference.

On GPUs, numerical results can sometimes vary between repeated runs because some operations are nondeterministic. Results can also differ across hardware, kernels, or precision settings: floating-point addition is not associative, so a different order of summation can change the last few bits. This is an implementation-level effect, not part of the mathematical algorithm. Even with greedy decoding, a sufficiently small numerical difference can flip the argmax between two closely tied tokens.

Parameter-count check

Summing every shape in Algorithm 10's parameter list (with Algorithm 5's attention parameters, $12 \times (3 \times (64{\times}768 + 64)) + 768{\times}768 + 768 = 23{,}62{,}368$ per layer, expanded in):

$$
\begin{aligned}
&\underbrace{768 \times 50257}_{W_e} + \\
&\underbrace{768 \times 1024}_{W_p} + \\
&12\,(\underbrace{23{,}62{,}368}_{\text{attn/layer}} + \underbrace{3{,}072}_{\text{LN/layer}} + \underbrace{47{,}22{,}432}_{\text{MLP/layer}}) + \\
&\underbrace{1{,}536}_{\text{final LN}} \\
&= 12{,}44{,}39{,}808
\end{aligned}
$$

with W_u tied to W_e (untying adds another $768{\times}50257 \approx 38.6$M, giving ≈ 163M). 124.4M matches the commonly cited "124M" parameter count for GPT2 small model.